1.1 Introduction to Node-JS

"Node's goal is to provide an easy way to build scalable network programs."
– Ryan Dahl, creator of Node.js

The World Wide Web (WWW) allows different types of content, such as text, images, videos, and other media, to connect and interact over the Internet. This is made possible through a standard set of communication protocols, with HTTP (Hypertext Transfer Protocol) being the most commonly used. These protocols define how data is transferred between web servers and clients (such as web browsers), allowing users to access and interact with a vast range of information and services.

As the Internet has grown, so too has the complexity of the applications we use. Modern web applications are no longer just simple static pages but are often sophisticated systems that deliver dynamic content to users. These applications are increasingly complex in both their functionality and the number of users they serve. As a result, there is a growing demand for technologies that can support the development, scaling, and management of such intricate networked applications.

Node.js is one such technology that has emerged to meet these challenges. By enabling non-blocking, event-driven programming, Node.js allows developers to build fast, efficient, and scalable network applications. It provides an excellent platform for creating web servers, real-time applications, APIs, and much more. Node's ability to handle many simultaneous connections with minimal resource usage has made it a popular choice for building high-performance applications.

Node.js is an open-source, cross-platform JavaScript runtime built on Google Chrome's V8 engine. It is widely used for building efficient and scalable applications. Node.js runs JavaScript outside the browser, allowing it to perform highly efficiently by using non-blocking, asynchronous I/O operations. This model enables Node.js to handle thousands of concurrent requests with a single process, unlike traditional servers, which block on I/O tasks like reading files or accessing networks.

Node.js was created in 2009 by Ryan Dahl to address the performance issues of traditional web servers, which would block when handling I/O tasks, reducing their throughput. By making I/O operations asynchronous, Node.js allows servers to continue processing other requests without waiting for previous ones to finish. This innovation made Node.js especially popular for handling real-time applications, APIs, and scalable network services.

One of Node.js's key advantages is that it allows JavaScript developers to write both client-side and server-side code, eliminating the need to learn a separate language for backend development. Additionally, Node.js supports the latest ECMAScript standards, and developers can use modern JavaScript features, and even experimental ones, by changing Node's version or using specific flags.

Node.js is also known for its low memory usage, speed, and the ability to create efficient, easy-to-manage servers with just a few lines of code. For example, a simple HTTP server in Node.js can be written in just 80 characters:

```
require('http').createServer((req, res) => res.end('Hello World')).listen(8080);
```

An Example Node.js Application

```
const { createServer } = require('node:http');
const hostname = '127.0.0.1';
const port = 3000;
const server = createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

1. const { createServer } = require('node:http');

This line imports the ***createServer*** function from the built-in http module in Node.js. The http module allows Node.js to create an HTTP server. Here, we're using JavaScript destructuring to directly get the ***createServer*** function from the module.

2. const hostname = '127.0.0.1';

This line defines a constant hostname with the value '127.0.0.1', which is the local IP address (also known as "localhost"). This means the server will only be accessible from the same machine it's running on.

3. const port = 3000;

This line defines a constant port with the value 3000. The port number represents the network endpoint through which the server will communicate.

4. const server = createServer((req, res) => { ... });

Here, we're calling ***createServer*** to create an HTTP server. The ***createServer*** function takes a callback function as its argument. This callback will be executed every time the server receives an HTTP request.

- req: The request object, which contains information about the incoming request (e.g., HTTP method, headers, URL).
- res: The response object, which is used to send the HTTP response back to the client.

Inside the callback, the code sets up the response for each incoming request.

5. res.statusCode = 200;

This line sets the status code of the HTTP response. In this case, 200 means "OK", which is a standard HTTP status code that indicates the request was successful.

6. res.setHeader('Content-Type', 'text/plain');

This line sets the Content-Type header for the response. Headers are part of the HTTP response and provide metadata about the response.

- 'Content-Type': 'text/plain': Specifies that the response body will be plain text (rather than, for example, HTML or JSON). This tells the client how to interpret the response content.

7. res.end('Hello World');

This line sends the response body back to the client and ends the response. The string 'Hello World' is the content that will be sent in the body of the response.

- res.end: This method is used to end the response process. The argument passed to res.end ('Hello World' in this case) will be sent as the response body.

8. server.listen(port, hostname, () => { ... });

This line starts the server and makes it listen for incoming requests on the specified port and hostname.

- port: The port on which the server will listen (in this case, 3000).
- hostname: The address at which the server will listen (in this case, '127.0.0.1', or localhost).
- The callback function inside listen will be executed once the server starts successfully, and it logs a message to the console.

9. console.log(Server running at http://${hostname}:${port}/);

This line logs a message to the console, confirming that the server is running. The message includes the hostname and port to tell you the exact address where the server is accessible.

- The message will be: "Server running at http://127.0.0.1:3000/".

1.2 Usage of Node.js

Node.js has emerged as a powerful tool for building modern, high-performance, and scalable applications. It is particularly well-suited for real-time applications, microservices architectures, APIs, and server-side applications. Below, we'll discuss several key use cases where Node.js excels, based on the problems it solves and its capabilities.

1. Real-Time Applications

Node.js is ideal for real-time applications, such as chat applications, gaming servers, and collaborative tools, due to its asynchronous, event-driven architecture. These applications often need to handle many users who are interacting in real-time, making it essential to handle thousands of concurrent connections without blocking.

2. API Servers

Node.js is often used to build APIs due to its non-blocking nature, making it an excellent choice for handling requests and serving responses quickly. APIs built with Node.js can handle high loads efficiently, making it ideal for microservices and other backend services.

3. Microservices Architecture

Node.js is particularly well-suited for building microservices, where different parts of an application are broken down into smaller, independently deployable services. The non-blocking nature of Node.js allows each microservice to handle requests efficiently, and it integrates well with other services in a distributed architecture.

4. Streaming Applications

Node.js can be used to build streaming applications, such as video or audio streaming platforms, because of its ability to handle large amounts of data without blocking. Its non-blocking I/O model allows it to stream media files efficiently to users without causing delays.

5. Command-Line Tools

Node.js can also be used to create command-line tools or scripts. Since Node.js has built-in modules for interacting with the file system, executing shell commands, and managing processes, it is an excellent choice for writing scripts and tools to automate tasks.

6. Serverless Applications

Node.js is widely used in serverless computing environments, such as AWS Lambda, where functions are executed in response to events (like HTTP requests). The lightweight and fast nature of Node.js makes it an ideal choice for serverless applications.

1.3 Advantages of using Node.js:

1. Non-blocking, Asynchronous I/O

Node.js is designed to handle asynchronous operations like reading files, querying databases, or handling network requests without blocking the main thread. This **non-blocking I/O** allows Node.js to handle many concurrent requests efficiently, without waiting for one operation to complete before starting another.

```
console.log('Start of program');
// Simulating an asynchronous task with setTimeout
setTimeout(() => {
  console.log('This message is printed after 2 seconds');
}, 2000);  // 2000 milliseconds = 2 seconds
console.log('End of program');
```

2. Single-Threaded Event Loop

Unlike traditional servers that spawn new threads for each request, Node.js runs on a **single-threaded event loop**. This means that all I/O operations are handled by a single thread, which simplifies the architecture and reduces the overhead caused by managing multiple threads.

```
console.log('Start of program');
// Simulate a non-blocking operation
setTimeout(() => {
  console.log('This runs after 2 seconds');
}, 2000);
// Simulate a synchronous operation
console.log('This is printed right after the first log');
// Simulate another non-blocking operation
setTimeout(() => {
  console.log('This runs after 1 second');
}, 1000);
console.log('End of program');
```

3. Scalability

Node.js is highly scalable, both vertically (adding more resources to a single server) and horizontally (adding more servers). Tools like **Node Cluster** allow you to spawn multiple processes to take full advantage of multi-core systems.

```
const cluster = require('cluster');
const http = require('http');
const os = require('os');
// Number of CPU cores on the machine
const numCPUs = os.cpus().length;
if (cluster.isMaster) {
  console.log(`Master process is running. Spawning ${numCPUs} workers...`);
  // Fork workers for each CPU core
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork(); // Fork a worker process for each CPU core
```

```
  }
  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died`);
  });
} else {
  // Worker process code
  http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Hello from Worker: ' + process.pid); // Send response with worker ID
  }).listen(8000);

  console.log(`Worker ${process.pid} is running`);
}
```

4. Fast Execution with V8 Engine

Node.js uses the **V8 JavaScript engine** (developed by Google for Chrome), which compiles JavaScript directly into native machine code. This enables Node.js to execute code very quickly and efficiently.

5. JavaScript on the Server-Side

With Node.js, developers can use **JavaScript on both the client-side and server-side**. This makes it easier for developers to work with both the frontend and backend of a web application without switching between languages.

6. Huge Ecosystem (npm)

Node.js has a massive ecosystem of open-source libraries available through **npm** (Node Package Manager). This ecosystem provides reusable modules and tools for everything from web frameworks (like Express) to database connectors and real-time communication libraries.

7. Real-Time Applications

Node.js excels at handling **real-time applications**, such as chat applications, live-streaming platforms, and collaborative tools, due to its event-driven, non-blocking architecture.

8. Cross-Platform Development

Node.js is **cross-platform**, meaning you can run it on multiple operating systems, such as Linux, Windows, and macOS, without modification to the code.

9. Large Community Support

Node.js has a large and active **community of developers** who contribute to its ecosystem and provide support through forums, blogs, and GitHub repositories.

10. Faster Development Cycle

Because of its lightweight, event-driven architecture, and the fact that you can use JavaScript throughout the entire application, **Node.js** can shorten the development cycle significantly. In addition, the **npm** ecosystem provides modules that further speed up development.

11. JSON and API Integration

Node.js uses **JSON** (JavaScript Object Notation) for handling data, which is a widely used format in web applications. This makes it a great choice for building **RESTful APIs** or integrating with external services.

12. Easy to Learn

Since Node.js is based on **JavaScript**, which is already widely known by developers, learning Node.js can be easier for web developers who are already familiar with JavaScript.

13. Microservices Architecture

Node.js fits very well into a **microservices architecture**, where applications are broken down into small, independent services that communicate with each other. Its lightweight nature and ease of scaling make it ideal for building microservices.

1.4 Disadvantage of Node.js

1. Single-Threaded Event Loop

- **Explanation**: Node.js operates on a single-threaded event loop model. This means it handles multiple requests using non-blocking I/O operations, but only one thread processes the requests at a time.
- **Disadvantages**:
  - **CPU-Intensive Tasks**: Node.js is not ideal for CPU-bound operations (like complex calculations, image processing, or heavy data computation). Since the event loop is single-threaded, heavy computations can block the thread, slowing down the processing of other requests.
  - **Limited Parallelism**: For handling parallel tasks, Node.js uses child processes or worker threads, which can add complexity to scaling the application.

2. Callback Hell (Nested Callbacks)

- **Explanation**: Node.js uses asynchronous callbacks to handle I/O operations. When multiple asynchronous operations are chained, the code can become deeply nested and difficult to manage.
- **Disadvantages**:
  - **Reduced Readability**: Deeply nested callbacks, often called "callback hell," can make code harder to read, understand, and maintain.
  - **Error Handling**: Proper error handling in nested callbacks is challenging, leading to more chances for bugs and hard-to-trace issues.

3. Not Ideal for Heavy Computation

- **Explanation**: Node.js is optimized for I/O-bound tasks, such as handling requests, reading files, and interacting with databases. However, it does not perform well for tasks that require significant CPU power.
- **Disadvantages**:
  - **Performance Bottleneck**: Heavy computational tasks (e.g., image/video processing or complex algorithms) will block the event loop, causing delays and affecting the overall application performance.
  - **Workarounds Required**: To mitigate this, you would need to use worker threads or delegate these tasks to separate services, which adds complexity.

4. Lack of Strong Typing

- **Explanation**: JavaScript, the language Node.js is built on, is dynamically typed. This means that variables do not have a fixed type and can change during runtime.
- **Disadvantages**:
  - **Runtime Errors**: Errors related to type mismatches or uninitialized variables only show up at runtime, making debugging more difficult.
  - **Harder Maintenance**: As the codebase grows, maintaining and understanding the code can become more difficult due to the absence of strict type definitions, unless you use TypeScript or other solutions.

5. Immaturity of Some Libraries

- **Explanation**: While Node.js has a vast ecosystem of packages (via npm), not all libraries are well-maintained or thoroughly tested.

- **Disadvantages**:
  - **Quality Concerns**: Some npm packages are of poor quality, outdated, or insecure. This can result in bugs, vulnerabilities, and difficulty in finding reliable dependencies.
  - **Dependency Bloat**: Over-reliance on npm packages can introduce unnecessary complexity and large file sizes, impacting performance.

6. Scalability Challenges

- **Explanation**: While Node.js is generally considered scalable due to its asynchronous nature, it can be tricky to manage at very large scales.
- **Disadvantages**:
  - **State Management**: Node.js is stateless by design, meaning that managing user sessions or state across multiple instances (e.g., for load balancing or clustering) requires additional infrastructure.
  - **Complex Scaling**: While the cluster module allows for forking multiple Node.js instances, managing these instances, load balancing, and maintaining high availability can be complex in large-scale applications.

7. Security Risks

- **Explanation**: Like any other platform, Node.js has security risks, particularly due to the large number of third-party libraries available via npm.
- **Disadvantages**:
  - **Vulnerabilities in Dependencies**: The reliance on npm packages can expose applications to vulnerabilities if these packages are outdated or poorly maintained.
  - **Insecure Coding Practices**: JavaScript is relatively easy to learn, and developers without a solid understanding of security best practices may introduce vulnerabilities like Cross-Site Scripting (XSS) or SQL injection.
  - **Lack of Built-In Security Features**: Node.js doesn't provide many built-in security features (like authentication and encryption) out of the box, requiring developers to implement their own or rely on third-party solutions.

8. Memory Consumption

- **Explanation**: Node.js is not optimized for heavy memory consumption tasks. It can struggle with large applications that consume significant memory or handle large files.
- **Disadvantages**:
  - **Memory Leaks**: Poorly written code or unoptimized packages can lead to memory leaks, which may impact the performance of long-running Node.js applications.
  - **Garbage Collection Overhead**: The V8 garbage collector may not always be efficient for memory-heavy applications, leading to performance degradation over time.

9. Limited Support for Multi-Threading

- **Explanation**: Node.js is single-threaded by design, and while it can spawn child processes, true multi-threading is not natively supported.
- **Disadvantages**:
  - **Concurrency Limitations**: Node.js cannot fully leverage multi-core CPUs for CPU-bound tasks unless you use worker threads, the cluster module, or external services, which complicates the development.

- **Not Ideal for Multithreaded Applications**: In contrast to languages like Java or C#, Node.js may not be the best choice for applications that require true multithreading, such as complex simulations or high-performance data processing.

10. Debugging Challenges

- **Explanation**: Debugging asynchronous code in Node.js, especially when using callbacks or promises, can be challenging.
- **Disadvantages**:
  - **Harder Debugging**: Tracing errors through multiple asynchronous operations or interdependent services (especially in microservices architectures) can become complex.
  - **Limited Debugging Tools**: While Node.js has some debugging tools, they are not as robust or feature-rich as the debugging tools available for more established languages like Java or Python.

11. Lack of Standardized Library for Heavy Computing

- **Explanation**: For applications that require scientific computing, AI, or machine learning, JavaScript (and by extension, Node.js) lacks mature libraries compared to languages like Python.
- **Disadvantages**:
  - **Limited Support**: Node.js is not ideal for use cases involving AI, machine learning, or other computational-heavy tasks, as the ecosystem is still relatively underdeveloped for these domains.

1.5 Real-World Examples

Node.js is a powerful runtime environment that enables developers to build scalable and fast applications, particularly for real-time, high-concurrency, and I/O-bound tasks. Below are some **use cases** and **real-world examples** of where Node.js is commonly used:

1. Real-Time Applications

- **Use Case**: Node.js is well-suited for applications that require real-time, two-way communication between the client and server. This includes applications like chat apps, live notifications, collaborative tools, and online gaming.
- **Real-World Example**:
  - **Socket.io**: Socket.io is a popular library built on Node.js that enables real-time, bidirectional communication between clients and servers. It's used in applications like:
    - **Trello**: The project management tool uses Node.js for real-time notifications and collaboration features where users can instantly see updates as they happen.
    - **Slack**: The messaging platform relies on Node.js to provide real-time messaging, allowing users to exchange messages instantly across various devices.

2. API Servers and Microservices

- **Use Case**: Node.js is often used to build lightweight, fast, and efficient API servers or microservices that can handle a large number of concurrent requests with minimal overhead.
- **Real-World Example**:

- **Netflix**: Netflix uses Node.js to build its server-side components. Its microservices architecture, implemented with Node.js, is designed to handle millions of concurrent requests for content delivery and real-time data.
- **Uber**: The ride-sharing giant uses Node.js for its real-time processing of requests, such as trip statuses, location tracking, and real-time pricing algorithms, because of Node.js's ability to handle high concurrency efficiently.

3. Streaming Applications

- **Use Case**: Applications that involve real-time data streaming, such as video, audio, or live updates, can leverage Node.js for efficient data transmission and processing.
- **Real-World Example**:
  - **Spotify**: Spotify uses Node.js for handling real-time music streaming, where a large number of users are continuously streaming music. Node.js's ability to handle a high volume of concurrent I/O operations makes it ideal for this use case.
  - **YouTube Live**: YouTube uses Node.js for its live-streaming features, allowing users to broadcast in real-time. The non-blocking I/O model helps in managing the heavy volume of data streaming without affecting performance.

4. Single-Page Applications (SPAs)

- **Use Case**: Node.js is commonly used in the backend of Single-Page Applications (SPAs) due to its ability to handle numerous API calls concurrently and provide fast response times.
- **Real-World Example**:
  - **LinkedIn**: The social networking site for professionals uses Node.js to serve its backend for the mobile app, providing a smooth user experience and handling high traffic volumes efficiently.
  - **PayPal**: PayPal adopted Node.js for their front-end and back-end services to build a fast, scalable architecture for their Single Page Application (SPA). They saw a performance boost due to Node's asynchronous nature.

5. E-commerce Websites

- **Use Case**: E-commerce platforms require real-time data updates, user interactions, and the ability to scale to handle large volumes of transactions. Node.js is ideal for managing high traffic, payment gateways, and inventory updates.
- **Real-World Example**:
  - **eBay**: eBay uses Node.js to handle millions of transactions daily. It helps in building real-time auction-style features and is used to serve the application with high scalability.
  - **Walmart**: Walmart used Node.js for its online retail platform to handle traffic spikes, particularly during sales events like Black Friday. It allowed them to scale their infrastructure effectively.

6. Internet of Things (IoT)

- **Use Case**: Node.js is often used in IoT applications due to its ability to handle multiple concurrent connections efficiently. This makes it well-suited for scenarios where many devices need to communicate with a central server or gateway.
- **Real-World Example**:

- **NASA**: NASA uses Node.js to handle real-time data from a wide variety of sensors and devices. Node.js's event-driven, non-blocking I/O model is ideal for collecting and processing large amounts of sensor data in real time.
- **Smart Home Systems**: Node.js is used in smart home platforms like **Nest**, where it processes the data coming from various smart devices and interacts with other systems via APIs to enable automation and real-time control.

7. Real-Time Collaboration Tools

- **Use Case**: Collaborative platforms, where multiple users can work together in real-time (e.g., document editing, project management), can benefit from Node.js's asynchronous nature and real-time communication features.
- **Real-World Example**:
  - **Google Docs**: While Google Docs is not fully built on Node.js, it uses similar real-time technologies for collaborative document editing. Node.js or similar real-time frameworks help deliver a seamless, live collaboration experience between users.
  - **Trello**: As a project management tool, Trello utilizes Node.js to handle real-time updates when multiple users make changes to boards or cards, ensuring that all users see the same information simultaneously.

8. Chat Applications

- **Use Case**: Chat applications require real-time, bidirectional communication between clients and servers. Node.js is ideal for building scalable chat systems with minimal latency.
- **Real-World Example**:
  - **WhatsApp Web**: The web version of WhatsApp uses Node.js to handle real-time messaging. The event-driven architecture of Node.js allows it to quickly push updates between users in real-time.
  - **Facebook Messenger**: Facebook uses Node.js to power its real-time messaging system, providing a responsive and fast experience for millions of concurrent users.

9. Microservices Architecture

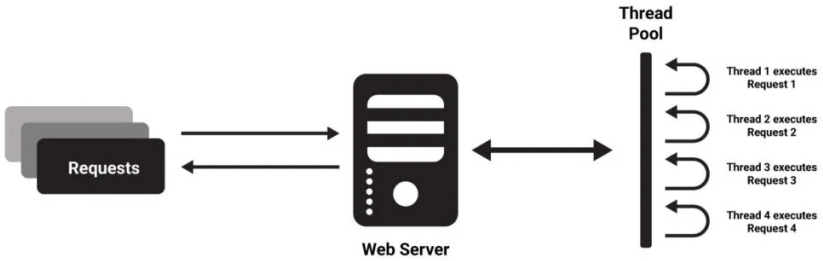
- **Use Case**: Node.js is ideal for building microservices, especially when you need lightweight services that can handle HTTP requests, respond quickly, and scale easily.
- **Real-World Example**:
  - **Netflix**: Netflix adopted a microservices architecture with Node.js for handling various backend services like user authentication, content management, and video streaming. The ability to scale individual services efficiently is a key advantage of Node.js in this architecture.
  - **Uber**: Uber relies on Node.js for building highly scalable microservices to process requests like ride status updates, driver locations, and fare calculations in real-time.

10. Server-Side Web Applications

- **Use Case**: Traditional web applications that require server-side logic for processing form submissions, handling requests, and serving dynamic content are a natural fit for Node.js.
- **Real-World Example**:

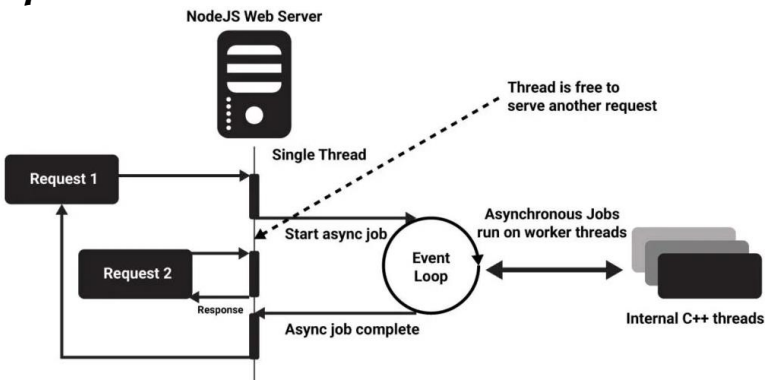
- **GoDaddy**: GoDaddy, a web hosting and domain registration company, uses Node.js to power several aspects of its web infrastructure, including web applications for managing customer accounts, hosting services, and email solutions.
- **Medium**: The publishing platform Medium uses Node.js on the backend to handle the large number of requests from writers and readers, processing data quickly and efficiently.

1.6 Traditional Web Server Model



In the traditional web server model, each request is handled by a dedicated thread from the thread pool. If no thread is available in the thread pool at any point of time then the request waits till the next available thread. Dedicated thread executes a particular request and does not return to thread pool until it completes the execution and returns a response.

1.7 Node.js process model



Node.js operates using an event-driven, non-blocking I/O model, making it lightweight and efficient. This model is built on three key architectural features:

1. Single-Threaded Event Loop

- Node.js runs on a single thread, handling all incoming requests on this main thread. While the synchronous part of each request is processed by the main thread, the asynchronous part is offloaded to background threads (k Workers).
- Events and emitters are central to Node.js, where actions (events) are queued and executed based on predefined conditions. The event loop processes these events, executing queued tasks as the main thread remains free for other requests.

2. Non-Blocking I/O Model

- In Node.js, non-blocking operations do not block the main thread. When an I/O operation (like reading a file or querying a database) is requested, Node.js offloads it to background threads, allowing the main thread to process other requests.
- Asynchronous I/O is handled efficiently, preventing delays and enabling concurrent execution without waiting for operations to complete.

3. Event-Driven and Asynchronous by Default

- Node.js relies on events and callbacks to manage asynchronous tasks. Once a background thread completes a task, it emits an event to notify the main thread, which then processes the callback.
- This design allows for concurrency and better resource management. The underlying **libuv** library handles most of Node.js's asynchronous I/O operations, ensuring efficient processing.

Together, these features enable Node.js to handle large numbers of concurrent requests with minimal resource consumption, making it ideal for real-time, I/O-heavy applications.

1.8 Traditional Web Server Model vs. Node.js Process Model

Feature	**Traditional Web Server (e.g., Apache, Nginx)**	**Node.js**
Concurrency Model	Multi-threaded (one thread/process per request)	Single-threaded event loop (asynchronous)
Request Handling	Blocking I/O (requests block until response is done)	Non-blocking I/O (asynchronous requests)
Performance	Well-suited for CPU-bound tasks, struggles with high concurrency	Excellent for I/O-bound tasks, struggles with CPU-bound tasks
Scalability	Limited by number of threads/processes, hard to scale	Scales efficiently for I/O-bound tasks, can scale horizontally
Resource Usage	High memory and CPU usage with many concurrent requests	Low memory and CPU usage, efficient with many concurrent requests
Suitability	Best for CPU-heavy or synchronous applications	Best for real-time, I/O-heavy, concurrent applications
Fault Tolerance	Can handle crashes per process/thread individually	Single-threaded nature may cause issues if event loop is blocked
Complexity of Handling Concurrency	Managing threads is complex and resource-intensive	Simple concurrency model, but requires handling of asynchronous callbacks

1.9How Node.js works?

Node.js is a **cross-platform**, **open-source back-end** JavaScript runtime environment. Node.js allows developers to use JavaScript to **create** command-line tools and server-side scripting, which involves executing scripts on the server before sending the page to the user's device. The V8 JavaScript engine, the basic library of packages, and some binaries are all part of the Node.js project.

The event-driven model of Node.js allows for **asynchronous I/O**. These architecture choices are intended to improve the **scalability** of web applications that have a lot of input/output operations, as well as for **real-time Web applications**.

Threading: Node.js uses a single thread event loop and non-blocking I/O calls to accommodate tens of thousands of simultaneous connections without incurring the overhead of thread context switching. In Node.js, a thread pool **manages** the execution of concurrent tasks. The default number of threads in the **libuv** thread pool can be **increased** by developers. These threads are likely to be distributed through several cores by the server **operating system (OS)**.

V8: V8 is Google's JavaScript engine, written in C++. It compiles and executes JavaScript code inside of a VM (Virtual Machine). When a webpage loaded into Google Chrome demonstrates some sort of dynamic effect, like automatically updating a list or news feed, you are seeing JavaScript, compiled by V8, at work. While Node itself will efficiently manage I/O operations, its process object refers to the V8 runtime. As such, it is important to understand how to configure the V8 environment, especially as your application grows in size.

Package management: The package manager npm comes **pre-installed** for the Node.js server platform. It organizes the installation and maintenance of **third-party** Node.js applications by installing them from the npm **repository**.

Event loop: Node.js files with the operating system so that it can be notified of new connections and receive callbacks. Instead of processes or threads, Node.js scales by using an event loop. Unlike other event-driven servers, Node.js's event loop does not require explicit calling. When there are no further callbacks to be executed, Node.js leaves the event loop.

Native bindings: Node.js has a C-based API called N-API that can be used to build **loadable** (importable).node modules from C/C++ source code. Third parties have created open-source C/++ wrappers on top of the API to fix the issue, which only partially solves the problem. They simplify interfaces, but they can add confusion that maintainers must contend with as a side effect. About the fact that Node.js' main functionality is included in a JavaScript built-in library, C++ plugins can be used to extend the features and boost the performance of applications.

1.10 Install Node JS

Node.js can be installed in multiple ways on a computer. The approach used by you depends on the existing Node.js development environment in the system. There are different package installers for different environments.

Steps to Install Node on Windows

You can install Node.js by grabbing a copy of the source code and compiling the application. Another way of installing Node.js is by cloning the Node.js GIT repository in all three environments and then installing it on the system.

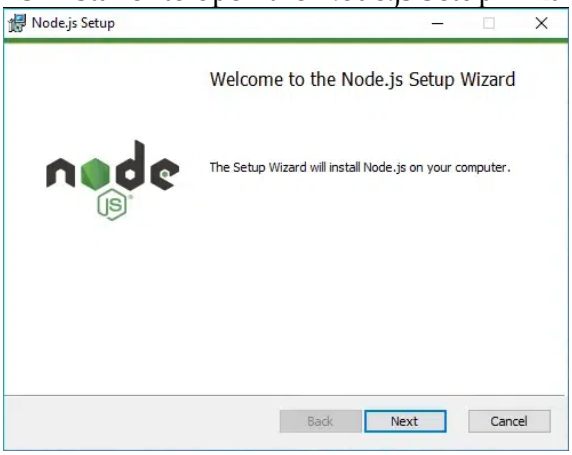
Step 1: Download NodeJS

Downloading the Node.js '.msi' installer the first step to install Node js on Windows is to download the installer. Visit the official Node.js website https://nodejs.org/en/download/

Step 2: Run the Node.js Installer

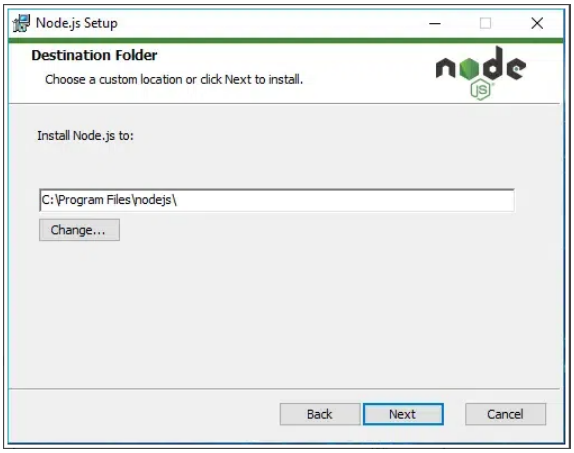
Once the Nodejs download is complete, you need to install the node.js installer on your PC. You need to follow the following steps for the Node.js to be installed:

2.1 Double-click on the .msi installer to open the Node.js Setup wizard and select "Next"

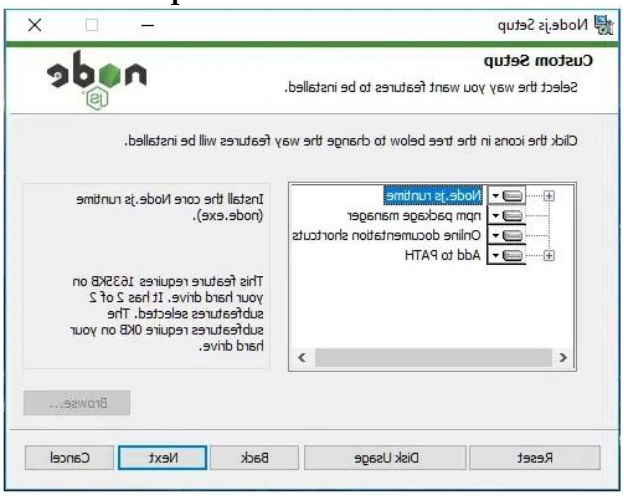


2.2 After clicking "Next", the End-User License Agreement (EULA) will open. Now, check "I accept the terms in the License Agreement" and select "Next"

2.3 Choose "Destination Folder" where you want to install Node.js & select "Next".

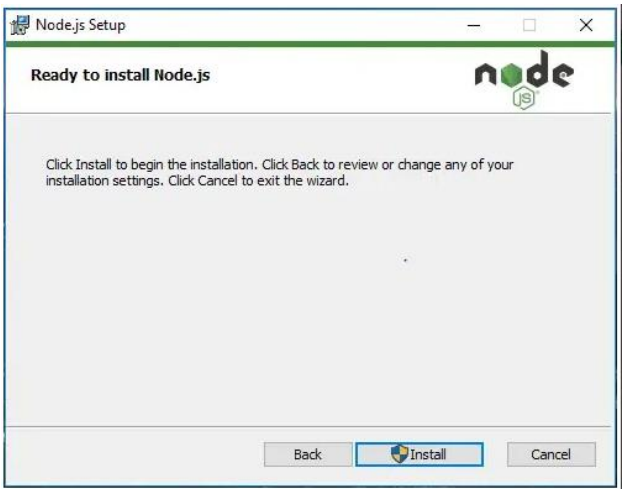


Else, go with this for **"Custom Setup"**.

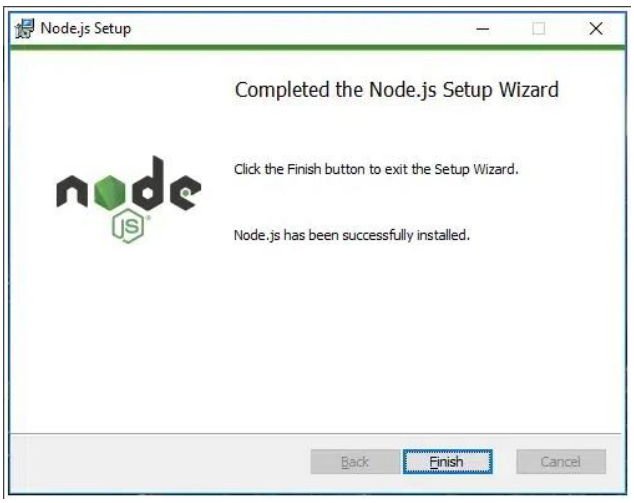


Step 3: Finish Setup and Install Node.js

The installer may prompt you to "install tools for native modules". Select "Install" to complete the process.



and wait for **"Finish"** to complete the setup.



3.1 Verify that Node.js was properly installed or not
To check whether node.js was completely installed on your system or not, you can run the following command in your command prompt or Windows Powershell and test it:
C:\Users\Admin> node –v

1.10.1 Working in REPL (Node.js interactive shell)

The Node.js *Read-Eval-Print-Loop* (REPL) is an interactive shell that processes Node.js expressions. The shell **reads** JavaScript code the user enters, **eval**uates the result of interpreting the line of code, **prints** the result to the user, and **loops** until the user signals to quit.
The REPL is bundled with every Node.js installation and allows you to quickly test and explore JavaScript code within the Node environment without having to store it in a file.
REPL stands for:

- **Read**: The REPL reads user input (JavaScript code).
- **Eval**: The input is evaluated (executed) by the JavaScript engine.
- **Print**: The result of the evaluation is printed to the screen.
- **Loop**: The process repeats (loops), waiting for the next input.

This interactive environment allows developers to test small snippets of code without needing to create a full script. The REPL is built directly into Node.js, so no additional setup is required.
Getting Started with REPL
To start working with REPL environment of NODE; open up the terminal (in case of UNIX/LINUX) or the Command prompt (in case of Windows) and write **node** and press 'enter' to start the REPL.
1. Executing JavaScript Code
The REPL is a full-featured JavaScript environment, meaning you can run any valid JavaScript code inside it.
Example:

```
> const x = 10;
> const y = 20;
> x + y
```

```
30
```

2. Multi-Line Input

In case of complex logic (like loops or functions), the REPL supports multi-line input. When you enter a block of code, the REPL will continue reading input until the block is complete.

Example:

```
> function add(a, b) {
...   return a + b;
... }
> add(5, 10)
15
```

3. Underscore (_) Variable

The REPL provides a special variable _ (underscore) that stores the result of the last evaluated expression.

Example:

```
> 3 + 3
6
> _ * 2
12
```

4. Saving and Loading REPL Sessions

The REPL allows you to save the session output to a file and load previously saved sessions, making it easier to keep track of the code you've tested.

Saving a Session: To save your REPL session to a file, use the .save command:

```
> .save ./repl_session.js
```

Loading a Session: You can load the saved session into a new REPL session using .load:

```
> .load ./repl_session.js
```

5. Accessing Node.js Core Modules

The REPL environment allows you to load and use Node.js core modules, such as fs, path, http, etc., without needing to exit the REPL or write an entire script.

Example:

```
> const fs = require('fs');
> fs.readFileSync('test.txt', 'utf8');
```

6. Error Handling in REPL

The REPL is forgiving of errors and will display helpful error messages without crashing the entire session. This makes it a safe environment for testing.

Example:

```
  > console.log(foo);
ReferenceError: foo is not defined
```

Important commands and keys in REPL

Here are some important commands to remember in REPL.

Whenever you need help, remember you can just type .help to receive it.

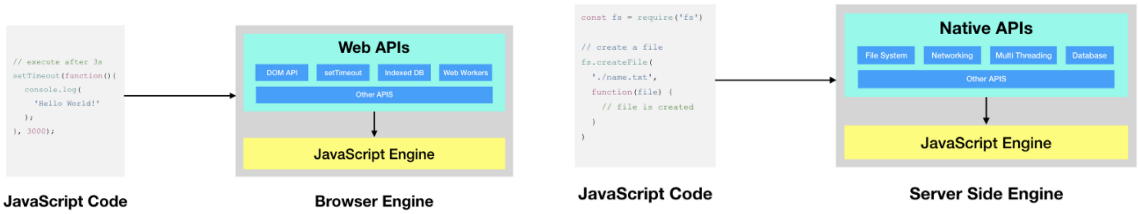
- **.exit**: Exit the REPL
- **.help**: Display help
- **.break**: Get out of current mode
- **.clear**: Alias for .break
- **.editor**: Enter editor mode

- **.load**: Load JavaScript from a file
- **.save**: Save all evaluated expression into a file
- **Ctrl + C**: Terminate the current command.
- **Ctrl + C (twice)**: Exit the REPL
- **Ctrl + D**: Exit the REPL
- **Tab Key**: Display a list of all commands.
- **Up / Down Keys**: Display previous commands

1.10.3 Node.js console and basic commands

Check Node.js version: To see which version of Node.js you're using, run: ***node –v***
Install Node.js packages with npm: Node.js uses npm (Node Package Manager) to manage packages. You can install packages from the npm registry with: ***npm install <package-name>***
Create a new Node.js project: In a new directory, initialize a Node.js project with npm init. This will create a package.json file.run: ***npm init***
Running a Node.js script: To execute a JavaScript file (e.g., app.js), use the node command: ***node app.js***
Install all dependencies: If you have a package.json file (e.g., after cloning a project), run: ***npm install***
List installed packages: To see all the packages installed locally in the project directory: ***npm list***
Global package installation: You can install packages globally with -g flag, making them available from anywhere on your system.Run ***npm install -g <package-name>***
Check npm version: To see which version of npm you have installed, run: ***npm -v***

1.11 How much JavaScript do you need to know to use Node.js?



1.11.1 JavaScript Fundamental

Lexical Structure: Understanding the syntax and structure of JavaScript code is fundamental for writing clean and concise code.

- **Case sensitivity**: JavaScript is case sensitive, which means that variables foo and Foo are considered to be different.
- **Whitespace**: JavaScript ignores most whitespace (e.g., spaces, tabs, and newlines), so you can use it to format your code in a way that makes it easier to read.
- **Line terminators**: JavaScript uses the newline character (\n) to indicate the end of a line.
- **Semicolons**: JavaScript uses semicolons (;) to separate statements. You can usually omit the semicolon if the statement is on a line by itself, but it is generally a good idea to use them consistently to avoid potential issues.
- **Comments**: JavaScript supports two types of comments: single-line comments (//) and multi-line comments (/* */).

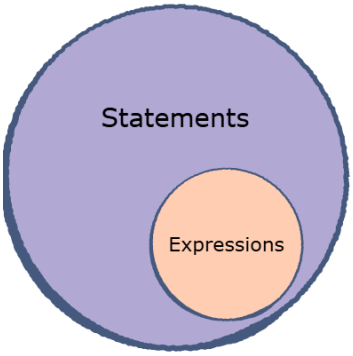
- **Identifiers**: Identifiers are used to name variables, functions, and other elements in the code. They can contain letters, digits, underscores (_), and dollar signs ($), and must begin with a letter, an underscore, or a dollar sign.
- **Keywords**: JavaScript has a set of reserved words that cannot be used as identifiers. These include words like var, function, and if.

1.11.2 JavaScript Expressions

JavaScript's expression is a valid set of literals, variables, operators, and expressions that evaluate a single value that is an expression. This single value can be a number, a string, or a logical value depending on the expression.

An **expression** is a block of code that evaluates to a value. A **statement** is any block of code that is performing some action.

The distinction between an expression and a statement is important because an expression is a subset of a statement. You can use an expression wherever a statement is expected, but this does not work vice versa.



There are five primary categories of expression in JavaScript:

- **Arithmetic**: uses arithmetic operators (+ - * / %).
- **String**: uses string operators and evaluates to a character string.
- **Logical**: evaluates to a boolean value of either *True* or *False* and uses boolean operators.
- **Primary expressions**: Consists of basic keywords and expressions.
- **Left-hand side expressions**: Left-hand side values which are the destination for the assignment operator.

1.11.3 JavaScript Variables and Datatypes

Variables and data types are foundational concepts in programming, serving as the building blocks for storing and manipulating information within a program. In JavaScript, getting a good grasp of these concepts is important for writing code that works well and is easy to understand.

Variables

In JavaScript, variables are used to store and manage data. They are created using the **var**, **let**, or **const** keyword.

1. ***var Keyword***

The var keyword is used to declare a variable. It has a function-scoped or globally-scoped behavior.

```
var a = "Hello Geeks"
var b = 10;
var c = 12;
```

```
var d = b + c;
console.log(a);
console.log(b);
console.log(c);
console.log(d);
```

2. ***let Keyword***

The let keyword in JavaScript is used to make variables that are scoped to the block they're declared in. Once you've used let to define a variable, you cannot declare it again within the same block. It's important to declare *let variables before using them*.

Syntax:

let variable_name = value;

1. Block Scope: The variables which are declared inside the { } block are known as block-scoped variables. variables declared by the var keyword cannot be block-scoped.

```
{
   let num = 10;
   // calling the function inside block
   console.log(num)
}

// Calling a function outside
// block throws an Error
console.log(num)
```

Output:

```
10
Uncaught ReferenceError: num is not defined
```

2. Global Scope :A global scope variable is a variable declared in the main body of the source code, outside all functions.

```
let num = 10;
console.log(num);
function fun() {
   console.log(num);
}
fun(); // Calling the function
```

Output:

```
10
10
```

3. Function Scope: A function scope variable is a variable declared inside a function and cannot be accessed outside the function.

```
function fun() {
   let num = 10;
   console.log(num);
}
fun(); //  Calling the function
console.log(num);
```

Output:

```
10
"ReferenceError: num is not defined
```

4. Redeclaring Variables in different blocks

The variables declared using let can be redeclared inside other blocks.

```
let x = 77;
{
   let x = 23;
   console.log(x);
}
console.log(x);
Output:
23
77
```

5. Redeclaring Variables in the same blocks

We cannot redeclare variables using the let keyword inside the same blocks. It will throw an error.

```
let x = 77;
{
   let x = 23; // legal
   console.log(x);
}
let x = 67; // illegal
console.log(x);
Output:
Uncaught SyntaxError: Identifier 'x' has already been declared
```

3. *const Keyword*

The **const keyword in JavaScript** is used to create variables that cannot be **redeclared or changed after their first assignment**. This keeps the variable's value fixed.

Additionally, const doesn't allow redeclaration of the same variable within the same block, and it provides block scope. It was introduced in ES2015 (ES6) for creating immutable variables.

Syntax:*const const_name;*

Characteristics of JavaScript const

Here are some Characteristics **of JavaScript Const :**

- **Cannot be reassigned**: Once a value is assigned to a const variable, it cannot be changed.
- **Block scope:** const is limited to the block in which it is defined, meaning it is not accessible outside of that block.
- **Must be assigned during declaration**: A const variable must be assigned a value at the time it is declared.
- **Works with primitive values:** const is often used with primitive values like numbers, strings, or booleans, making them immutable.
- **Objects and arrays can be modified**: While a const object or array cannot be reassigned to a new reference, the values or properties inside them can be modified.

- **Can't reference new objects or arrays**: You can modify the content of const arrays or objects but cannot reassign them to a completely new array or object.
- **Can be redeclared in different block scopes**: const variables can be declared again in a different block scope without any conflict.
- **Cannot be hoisted:** Unlike var, const variables are not hoisted and must be declared before use.
- **Creates read-only references:** const creates a reference to a value that cannot be changed, although properties of objects and arrays may still be altered.

Cannot Be Reassigned : Once a value is assigned to a const variable, it cannot be reassigned. Any attempt to change the value of a const variable will throw an error.

```
const x = 12;
x = 13;
x += 1;
//Uncaught TypeError: Assignment to constant variable.
```

Block Scope : Variables declared with const are limited to the block, statement, or expression in which they are defined. They cannot be accessed outside this block.

```
if (true) {
   const y = 10;
   console.log(y); // 10
}
console.log(y); // Error: y is not defined
//Uncaught SyntaxError: Uncaught ReferenceError: y is not defined
```

Variables Must Be Assigned on Declaration

A const variable must be initialized at the time of declaration. Declaring a const variable without assigning a value will result in a syntax error.

```
const x;
x = 12;
//Uncaught SyntaxError: Missing initializer in const declaration
```

Cannot Be Hoisted Like var

const variables are not hoisted in the same way as var. They cannot be used before they are declared, or it will result in an error.

```
x = 3;
console.log(x);
const x;
//Uncaught SyntaxError: Missing initializer in const declaration
```

Example Of Const :

1. const in Arrays

```
// Changing the content of array is
// possible in cost array
const arr1 = ["pankaj", "sumit", "chandan", "ajay"];
console.log(arr1.toString());
arr1[2] = "Narayan"; // possible
console.log(arr1.toString());
```

2. const in Objects

```
const person = {
```

```
  first_name: "Pankaj",
  last_name: "Singh",
  Age: 20,
  About: "Web Developer and Competitive Programmer"
};

console.log(person);

// It is possible
person.first_name = "Aryan";
person.last_name = "Yadav";
person.Age = 22;
person.About = "Commerce undergraduate";

console.log(person);

// it is not possible
// const person={
// "first_name":"Aryan",
// "last_name":"Yadav",
// "Age":22,
// "About":"Commerce undergraduate"
// }
```

Limitations of const

While const is powerful for creating stable, immutable references, there are a few limitations:

- **No Percentage Units**: Only length-based values like px, em, or rem can be used in property settings; percentages are not applicable.
- **Incompatibility with Justified Text**: const variables cannot be used effectively with justified text as they may introduce uneven spacing.
- **Cross-Browser Consistency**: Different browsers may show slight differences in rendering, potentially affecting layout

Data Types

JavaScript is a **dynamically typed** (also called loosely typed) scripting language. In JavaScript, variables can receive different data types over time. The latest ECMAScript standard defines eight data types Out of which seven data types are **Primitive (predefined)** and one **complex or Non-Primitive**.

Some Fact about JavaScript DataType

Dynamically Typed : JavaScript Variables are not bound to a specific data type. Mainly data type is stored with value (not with variable name) and is decided & checked at run time.

```
let x = 42;
console.log(x)
x = "hello";
console.log(x)
x = [1, 2, 3]
console.log(x)
```

Everything is an Object (Sort of): In JavaScript, Functions are objects, arrays are objects, and even primitive values can behave like objects temporarily when you try to access properties on them.

```
let s = "hello";
console.log(s.length);

// Example with a number
let x = 42;
console.log(x.toString());

// Example with a boolean
let y = true;
console.log(y.toString());

/* Internal Working of primitives   to be treeated as objects

// Temporary wrapper object
let temp = new String("hello");

console.log(temp.length); // 5

// The wrapper is discarded after use
temp = null; */
```

NaN is not equal to itself: NaN Stands for "Not-a-Number": It is used to represent a computational error. NaN is technically of type number.

```
console.log(typeof NaN);
console.log(NaN === NaN);
Output:false
```

A Symbol is Never Equal to Another One : Symbol is a unique and immutable data type often used for creating private properties and methods. Symbols are never equal to any other Symbol.

```
let s1 = Symbol("abc");
let s2 = Symbol("abc");
console.log(s1 === s2);
Output:  false
```

Undefined and Null: undefined represents a variable that has been declared but not assigned, while null is an explicit assignment representing "no value".

Integers are Floating are Numbers only. There is only one type number that covers both integers and floating point numbers.

```
let x = 42;     // Integer
let y = 42.5;   // Floating-point

console.log(typeof x);
console.log(typeof y);
Output
```

```
number
number
```

A character is also a string. There is no separate type for characters. A single character is also a string.

```
let s1 = "gfg";   // String
let s2 = 'g';    // Character
console.log(typeof s1);
console.log(typeof s2);
Output
string
string
```

JavaScript Primitive Data Type

1. Number

The Number data type represents both integer and floating-point numbers.

The Number data type contains both integer and floating-point numbers. JavaScript does not distinguish between integers and floats. Some special numbers in JavaScript are: 'Infinity', '-Infinity', and 'NaN'. The 'NaN' denotes a computational error.

```
let n1 = 2;
console.log(n1)
let n2 = 1.3;
console.log(n2)
let n3 = Infinity;
console.log(n3)
let n4 = 'something here too'/2;
console.log(n4)
Output:
2
1.3
Infinity
NaN
```

2. String

A String in JavaScript is a series of characters that are surrounded by quotes. There are three types of quotes in Javascript, which are:

```
let s = "Hello There";
console.log(s);

let s1 = 'Single quotes work fine';
console.log(s1);

let s2 = `can embed ${s}`;
console.log(s2);
Output:
Hello There
Single quotes work fine
can embed Hello There
```

There's no difference between 'single' and "double" quotes in JavaScript. Backticks provide extra functionality as with their help of them we can embed variables inside them.

3. Boolean

The boolean type has only two values i.e. **true** and **false**.

```
let b1 = true;
console.log(b1);

let b2 = false;
console.log(b2);
```

Output

```
true
false
```

4. Null

The special null value does not belong to any of the default data types. It forms a separate type of its own which contains only the null value.

```
let age = null;
console.log(age)
```

Output

```
null
```

The 'null' data type defines a special value that represents nothing, or empty value.

5. Undefined

A variable that has been declared but not initialized with a value is automatically assigned the undefined value. It means the variable exists, but it has no value assigned to it.OU

```
let a;
console.log(a);
```

Output

```
undefined
```

6. Symbol (Introduced in ES6)

Symbols are new primitive data types introduced as part of ES6. A Symbol is a unique and immutable primitive value that is used as an identifier for object properties. Symbols are primarily used for creating unique property keys in objects, ensuring there are no conflicts with other properties.

7. BigInt (Introduced in ES2020)

BigInt is a built-in object that provides a way to represent whole numbers greater than 253. The largest number that JavaScript can reliably represent with the Number primitive is 253, which is represented by the MAX_SAFE_INTEGER constant.

```
let b = BigInt("0b1010101001010101001111111111111111");
console.log(b);
```

JavaScript Non-Primitive Data Types

1.Object

JavaScript objects are fundamental data structures used to store collections of data. They consist of key-value pairs and can be created using curly braces {} or the new keyword. Understanding objects is crucial, as everything in JavaScript is essentially an object.

2. Arrays

An Array is a special kind of object used to store an ordered collection of values, which can be of any data type.

1.11.4 Loops and Control in JavaScript

Control structures are fundamental building blocks in programming that allow developers to control the flow of execution in their code. In JavaScript, control structures such as if statements, switch statements, and loops (for, while, do-while) play a crucial role in determining the behavior of a program.

1) If Statement:

The if statement is a fundamental control structure in JavaScript used to execute a block of code if a specified condition evaluates to true.

```
let num = 10;
if (num > 0) {
   console.log("Number is positive");
}
```

2) If-Else Statement:

The if-else statement allows us to execute one block of code if the condition is true and another block if the condition is false.

```
let num = -5;
if (num > 0) {
   console.log("Number is positive");
} else {
   console.log("Number is not positive");
}
```

3) Switch Statements:

Switch statements provide an efficient way to handle multiple conditions by evaluating an expression and executing the corresponding case.

```
let day = "Monday";
switch (day) {
   case "Monday":
      console.log("Today is Monday");
      break;
   case "Tuesday":
      console.log("Today is Tuesday");
      break;
   default:
      console.log("Unknown day");
}
```

4) Ternary Operator (Conditional Operator):

The ternary operator is a concise way to write if-else statements in a single line

Syntax: The ternary operator has the following syntax:

```
condition ? expression1 : expression2
```

5) JavaScript for Loop

The JS for loop provides a concise way of writing the loop structure. The for loop contains initialization, condition, and increment/decrement in one line thereby providing a shorter, easy-to-debug structure of looping.

```
for (initialization; testing condition; increment/decrement) {
   statement(s)
}
```

6) JavaScript while Loop

The JS while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

```
while (boolean condition) {
   loop statements...
}
```

7) JavaScript do-while Loop

The JS do-while loop is similar to the while loop with the only difference is that it checks for the condition after executing the statements, and therefore is an example of an **Exit Control Loop.** It executes loop content at least once event the condition is false.

```
do {
   Statements...
}
while (condition);
```

8) JavaScript for-in Loop

JS for-in loop is used to iterate over the properties of an object. The for-in loop iterates only over those keys of an object which have their enumerable property set to "true".

```
for(let variable_name in object_name) {
   // Statement
}
```

Example:

```
let myObj = { x: 1, y: 2, z: 3 };
for (let key in myObj) {
   console.log(key, myObj[key]);
}
```

5. JavaScript for-of Loop

JS for-of loop is used to iterate the iterable objects for example – array, object, set and map. It directly iterate the value of the given iterable object and has more concise syntax than for loop.

```
for(let variable_name of  object_name) {
   // Statement
}
```

Example:

```
let arr = [1, 2, 3, 4, 5];
for (let value of arr) {
   console.log(value);
}
```

1.11.5 Functions in JavaScript

A function in JavaScript is a reusable block of code that performs a specific task. You define it once, and then you can run (or "call") it whenever you need that task done in your program.

Function Declaration:

```
function functionName(parameter1, parameter2) {
```

```
  // Code to be executed
  return result; // Optional
}
```

Exmple:

```
function sum(x, y) {
  return x + y;
}
console.log(sum(6, 9));
```

Function Syntax and Working

A function definition is sometimes also termed a function declaration or function statement. Below are the rules for creating a function in JavaScript:

- Begin with the keyword ***function*** followed by,
- A user-defined function name (In the above example, name is **sum**)
- A list of parameters enclosed within parentheses and separated by commas (In the above example, parameters are **x** and **y**)
- A list of statements composing the body of the function enclosed within curly braces {} (In the above example, statement is "return x + y").

Return Statement : There are some situations when we want to return some values from a function after performing some operations. In such cases, we make use of the return. This is an optional statement. In the above function, "sum()" returns the sum of two as result.

Function Parameters : Parameters are input passed to a function. In the above example, sum() takes two parameters, x and y.

Calling Functions: After defining a function, the next step is to call them to make use of the function. We can call a function by using the function name separated by the value of parameters enclosed between the parenthesis.

Function Expression: It is similar to a function declaration without the function name. Function expressions can be stored in a variable assignment.

Syntax:

```
let javascriptforBWU= function(paramA, paramB) {
  // Set of statements
}
```

Example:

```
const square = function (number) {
    return number * number;
};
const x = square(4); // x gets the value 16
console.log(x);
```

Arrow Function is one of the most used and efficient methods to create a function in JavaScript because of its comparatively easy implementation. It is a simplified as well as a more compact version of a regular or normal function expression or syntax.

Syntax:

```
let function_name = (argument1, argument2 ,..) => expression
```

Example:

```
const divide = (x, y) => x / y;
```

```
console.log(divide(10, 2)); // Outputs: 5
```

1. Arrow Function without Parameters

An arrow function without parameters is defined using empty parentheses (). This is useful when you need a function that doesn't require any arguments.

```
const gfg = () => {
   console.log( "Hi from GeekforGeeks!" );
}
gfg();
```

2. Arrow Function with Single Parameters

If your arrow function has a single parameter, you can omit the parentheses around it. returning the square of x.

```
const square = x => x*x;
console.log(square(4));
// output: 16
```

3. Arrow Function with Multiple Parameters

Arrow functions with multiple parameters, like **(param1, param2) => { }**, simplify writing concise function expressions in JavaScript, useful for functions requiring more than one argument.

```
const gfg = ( x, y, z ) => {
   console.log( x + y + z )
}

gfg( 10, 20, 30 );
```

4. Arrow Function with Default Parameters

Arrow functions support default parameters, allowing predefined values if no argument is passed, making JavaScript function definitions more flexible and concise.

```
const gfg = ( x, y, z = 30 ) => {
   console.log( x + " " + y + " " + z);
}
gfg( 10, 20 );
```

5. Return Object Literals

In JavaScript, returning object literals within functions is concise: **() => ({ key: value })** returns an object { key: value }, useful for immediate object creation and returning.

```
const makePerson = (firstName, lastName) =>
({first: firstName, last: lastName});
console.log(makePerson("Pankaj", "Bind"));
```

1.11.6 JavaScript Arrays

An array in JavaScript is a data structure used to store multiple values in a single variable. It can hold various data types and allows for dynamic resizing. Elements are accessed by their index, starting from 0.

1. Create using Literal

Creating an array using array literal involves using square brackets [] to define and initialize the array.

```
// Creating an Empty Array
let a = [];
```

```
console.log(a);
// Creating an Array and Initializing with Values
let b = [10, 20, 30];
console.log(b);
```

2. Create using new Keyword (Constructor)

The "**Array Constructor**" refers to a method of creating arrays by invoking the Array constructor function.

```
// Creating and Initializing an array with values
let a = new Array(10, 20, 30);
console.log(a);
```

Basic Operations on JavaScript Arrays

1. Accessing Elements of an Array

Any element in the array can be accessed using the index number. The index in the arrays starts with 0.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Accessing Array Elements
console.log(a[0]);
console.log(a[1]);
```

2. Accessing the First Element of an Array

The array indexing starts from 0, so we can access first element of array using the index number.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Accessing First Array Elements
let fst = a[0];
console.log("First Item: ", fst);
```

3. Accessing the Last Element of an Array

We can access the last array element using [array.length – 1] index number.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Accessing Last Array Elements
let lst = a[a.length - 1];
console.log("First Item: ", lst);
let fruits = ["Apple", "Orange", "Plum"];
// same as fruits[fruits.length-1]
alert( fruits.at(-1) ); // Plum
```

4. Modifying the Array Elements

Elements in an array can be modified by assigning a new value to their corresponding index.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
console.log(a);
a[1]= "Bootstrap";
console.log(a);
```

5. Adding Elements to the Array

Elements can be added to the array using methods like push() and unshift().

- The push() method add the element to the end of the array.
- The unshift() method add the element to the starting of the array.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Add Element to the end of Array
a.push("Node.js");
// Add Element to the beginning
a.unshift("Web Development");
console.log(a);
```

6. Removing Elements from an Array

To remove the elements from an array we have different methods like pop(), shift(), or splice().

- The pop() method removes an element from the last index of the array.
- The shift() method removes the element from the first index of the array.
- The splice() method removes or replaces the element from the array.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
console.log("Original Array: " + a);
// Removes and returns the last element
let lst = a.pop();
console.log("After Removing the last: " + a);
// Removes and returns the first element
let fst = a.shift();
console.log("After Removing the First: " + a);
// Removes 2 elements starting from index 1
a.splice(1, 2);
console.log("After Removing 2 elements starting from index 1: " + a);
```

7. Array Length

We can get the length of the array using the array length property.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
let len = a.length;
console.log("Array Length: " + len);
```

8. Increase and Decrease the Array Length

We can increase and decrease the array length using the JavaScript length property.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"]
// Increase the array length to 7
a.length = 7;
console.log("After Increasing Length: ", a);
// Decrease the array length to 2
a.length = 2;
console.log("After Decreasing Length: ", a)
```

A word about "length"

The length property automatically updates when we modify the array. To be precise, it is actually not the count of values in the array, but the greatest numeric index plus one.

9. Iterating Through Array Elements

We can iterate array and access array elements using for loop and forEach loop.

Example: It is an example of for loop.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Iterating through for loop
for (let i = 0; i < a.length; i++) {
   console.log(a[i])
}
```

Example: It is the example of Array.forEach() loop.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Iterating through forEach loop
a.forEach(function myfunc(x) {
   console.log(x);
});
```

10. Array Concatenation

Combine two or more arrays using the concat() method. It returns new array containing joined arrays elements.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS", "React"];
let b = ["Node.js", "Expess.js"];
// Concatenate both arrays
let concateArray = a.concat(b);
console.log("Concatenated Array: ", concateArray);
```

11. Conversion of an Array to String

We have a builtin method **toString()** to converts an array to a string.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Convert array ot String
console.log(a.toString());
```

12. Check the Type of an Arrays

The JavaScript typeof operator is used ot check the type of an array. It returns "object" for arrays.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Check type of array
console.log(typeof a);
```

13. Recognizing a JavaScript Array

There are two methods by which we can recognize a JavaScript array:

- **By using Array.isArray() method**
- **By using instanceof method**

Below is an example showing both approaches:

```
const courses = ["HTML", "CSS", "Javascript"];
console.log("Using Array.isArray() method: ", Array.isArray(courses))
console.log("Using instanceof method: ", courses instanceof Array)
```

14. map in JavaScript

The arr.map method is one of the most useful and often used.
It calls the function for each element of the array and returns the array of results.
The syntax is:

```
let result = arr.map(function(item, index, array) {
  // returns the new value instead of item
});
```

Example:

```
let lengths = ["Bilbo", "Gandalf", "Nazgul"].map(item => item.length);
alert(lengths); // 5,7,6
```

Example:

```
const array1 = [1, 4, 9, 16];
// Pass a function to map
const map1 = array1.map((x) => x * 2);
console.log(map1);
// Expected output: Array [2, 8, 18, 32]
```

15. filter in JavaScript

The filter() method of Array instances creates a shallow copy of a portion of a given array, filtered down to just the elements from the given array that pass the test implemented by the provided function.
The syntax is:

```
let results = arr.filter(function(item, index, array) {
  // if true item is pushed to results and the iteration continues
  // returns empty array if nothing found
});
```

Example:

```
const words = ['spray', 'elite', 'exuberant', 'destruction', 'present'];
const result = words.filter((word) => word.length > 6);
console.log(result);
// Expected output: Array ["exuberant", "destruction", "present"]
```

15. find in JavaScript

The **find()** method of Array instances returns the first element in the provided array that satisfies the provided testing function. If no values satisfy the testing function, undefined is returned.
The syntax is:

```
let result = arr.find(function(item, index, array) {
  // if true is returned, item is returned and iteration is stopped
  // for falsy scenario returns undefined
});
```

```
const array1 = [5, 12, 8, 130, 44];
const found = array1.find((element) => element > 10);
console.log(found);
```

```
// Expected output: 12
```

1.11.7 JavaScript Classes

In JavaScript, a **class** is a blueprint for creating objects with shared properties and methods. Introduced in ES6, classes provide a much more elegant and structured way to work with objects and inheritance than the older prototype-based approach.

```
class ClassName {
  constructor() {
    // Initialization code goes here
  }
  methodName() {
    // Method code goes here
  }
}
```

Example 1: Basic Class Definition

```
class Car {
  constructor(make, model) {
    this.make = make;
    this.model = model;
  }

  getDetails() {
    return `${this.make} ${this.model}`;
  }
}

const car1 = new Car('Toyota', 'Corolla');
console.log(car1.getDetails()); // Output: Toyota Corolla
```

Example 2: Class Inheritance

You can create a subclass (child class) that extends another class (parent class). The subclass inherits the properties and methods of the parent class.

```
class Vehicle {
  constructor(type) {
    this.type = type;
  }
  getType() {
    return this.type;
  }
}
class Car extends Vehicle {
  constructor(make, model, type) {
    super(type); // Call the parent class constructor
    this.make = make;
    this.model = model;
  }
  getDetails() {
```

```
    return `${this.make} ${this.model} - ${this.getType()}`;
  }
}
const myCar = new Car('Toyota', 'Corolla', 'Sedan');
console.log(myCar.getDetails()); // Output: Toyota Corolla – Sedan
```

Example 3: Getter and Setter Methods

```
class Person {
  constructor(name, age) {
    this._name = name;  // underscore to indicate private-like property
    this._age = age;
  }

  get name() {
    return this._name;
  }

  set name(value) {
    this._name = value;
  }

  get age() {
    return this._age;
  }

  set age(value) {
    if (value >= 0) {
      this._age = value;
    } else {
      console.log('Age cannot be negative');
    }
  }
}

const person = new Person('Alice', 30);
console.log(person.name); // Output: Alice
person.age = 35;
console.log(person.age);  // Output: 35
person.age = -5;          // Output: Age cannot be negative
```

Example 4: Static Methods

Static methods are called on the class itself, not on instances of the class.

```
class Calculator {
  static add(a, b) {
    return a + b;
  }
```

```
  static subtract(a, b) {
    return a - b;
  }
}

console.log(Calculator.add(5, 3));       // Output: 8
console.log(Calculator.subtract(5, 3));   // Output: 2
```

Example 5: Class with Private Fields (ES2022)

With ES2022, JavaScript now supports private fields in classes using #.

```
class Rectangle {
  #width;
  #height;

  constructor(width, height) {
    this.#width = width;
    this.#height = height;
  }

  getArea() {
    return this.#width * this.#height;
  }
}

const rect = new Rectangle(5, 10);
console.log(rect.getArea()); // Output: 50
// Trying to access private field directly would cause an error:
// console.log(rect.#width); // Error: Private field '#width' must be declared in an enclosing class
```

1.11.8 Objects in Javascript

An object in JavaScript is a data structure used to store related data collections. It stores data as key-value pairs, where each key is a unique identifier for the associated value. Objects are dynamic, which means the properties can be added, modified, or deleted at runtime.

1. Creation sing Object Literal

The object literal syntax allows you to define and initialize an object with curly braces {}, setting properties as key-value pairs.

```
let obj = {
   name: "Sourav",
   age: 23,
   job: "Developer"
};
console.log(obj);
```

Output

```
{ name: 'Sourav', age: 23, job: 'Developer' }
```

2. Creation Using new Object() Constructor

```
let obj = new Object();
```

```
obj.name= "Sourav",
obj.age= 23,
obj.job= "Developer"
console.log(obj);
```

Output

```
{ name: 'Sourav', age: 23, job: 'Developer' }
```

Basic Operations on JavaScript Objects

1. Accessing Object Properties

You can access an object's properties using either **dot notation** or **bracket notation**

```
let obj = { name: "Sourav", age: 23 };
// Using Dot Notation
console.log(obj.name);
// Using Bracket Notation
console.log(obj["age"]);
```

2. Modifying Object Properties

Properties in an object can be modified by reassigning their values.

```
let obj = { name: "Sourav", age: 22 };
console.log(obj);
obj.age = 23;
console.log(obj);
```

3. Adding Properties to an Object

You can dynamically add new properties to an object using dot or bracket notation.

```
let obj = { model: "Tesla" };
obj.color = "Red";
console.log(obj);
```

4. Removing Properties from an Object

The delete operator removes properties from an object.

```
let obj = { model: "Tesla", color: "Red" };
delete obj.color;
console.log(obj);
```

5. Checking if a Property Exists

You can check if an object has a property using the in operator or hasOwnProperty() method.

```
let obj = { model: "Tesla" };
console.log("color" in obj);
console.log(obj.hasOwnProperty("model"));
```

6. Iterating Through Object Properties

Use for…in loop to iterate through the properties of an object.

```
let obj = { name: "Sourav", age: 23 };
for (let key in obj) {
   console.log(key + ": " + obj[key]);
}
```

7. Merging Objects

Objects can be merged using Object.assign() or the spread syntax { …obj1, …obj2 }.

```
let obj1 = { name: "Sourav" };
let obj2 = { age: 23};
```

```
let obj3 = { ...obj1, ...obj2 };
console.log(obj3);
```

8. Methods in Objects

```
let car = {
  brand: 'Toyota',
  start() {
    console.log('Engine started');
  }
};
car.start(); // 'Engine started'
```

8. Object Length

You can find the number of properties in an object using **Object.keys().**

```
let obj = { name: "Sourav", age: 23 };
console.log(Object.keys(obj).length);
```

Recognizing a JavaScript Object

To check if a value is an object, use typeof and verify it's not null.

```
let obj = { name: "Sourav" };
console.log(typeof obj === "object" && obj !== null);
```

1.11.9 JavaScript this Keyword

In JavaScript, **this keyword** refers to the object that is currently executing a function or method. Its value is determined by how a function is called. this typically represents the context in which a function operates, allowing access to the properties and methods of its containing object.

1. Using this in a Method

In the context of an object method in JavaScript, the this keyword refers to the object itself, allowing access to its properties and methods within the method's scope. It facilitates interaction with the object's data and behavior, providing a way to access and manipulate its state.

```
const person = {
   name: 'John',
   age: 30,
   greet() {
      console.log('Hello, my name is ' +
         this.name + ' and I am '
         + this.age +
         ' years old.');
   }
};

person.greet();
```

2. Using this in a Function

In a JavaScript function, the behavior of the this keyword varies depending on how the function is invoked.

Syntax:

```
function exampleFunction() {
  console.log(this); // Refers to the current execution context
}
```

3. Using this alone

When used alone in JavaScript, outside of any specific context, the behavior of the this keyword depends on whether the code is running in strict mode or not.

4. Implicit Binding

When we call a function as a method of the object **this** keyword refers to the calling object

```
const person = {
   name: "ram",
   age: 22,
   greet: function () {
      return `Hello ${this.name}, you are ${this.age} years old`
   }
}
console.log(person.greet());
```

5. Explicit Binding

When we explicitly bind this keyword using the call(), bind(), or apply() method then this keyword default reference is changed to the object called using the above-specified methods.

```
function ageVerify(){
   if(this.age> 18){
      console.log("Yes you can drive");
   } else {
      console.log("No you cannot drive");
   }
}

const per1 = {age: 21};
const per2 = {age: 16};
ageVerify.call(per1);
ageVerify.call(per2);
```

6. Default Binding

When this keyword is used in global scope this is set to window object.

```
const age = 22;
function verifyAge (){
   return this.age;
}

console.log(verifyAge());
```

7. Arrow Function Binding

When this is used in the arrow function then this has lexical scope so without the function keyword this is unable to refer to the object in the outer scope.

```
const person = {
   name: "ram",
   age: 22,
```

```
  greet : () =>{
     return `Hello , you are ${this.age} years old`
  }
}
console.log(person.greet());
```

1.11.9 Asynchronous Programming in JavaScript

To understand what asynchronous programming means, think about multiple people working on a project simultaneously, each on a different task.

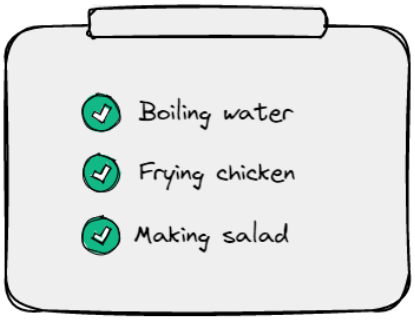
In traditional (synchronous) programming, each person would have to wait for the person before them to finish their task before starting their own.

But with asynchronous programming, everyone can start and work on their tasks simultaneously without waiting for the others to finish.

What is Synchronous Programming?

Synchronous programming is a way for computers to do things one step at a time, in the order they are given the instructions.

Imagine you're cooking dinner and have a list of tasks, like boiling water for pasta, frying chicken, and making a salad.



You would do these tasks one at a time and wait for each one to finish before moving to the next.

Synchronous programming works similarly, where the computer will complete one task before moving on to the next. This makes it easy to understand and predict what the computer will do at any given time.

Here's an example of synchronous code in JavaScript:

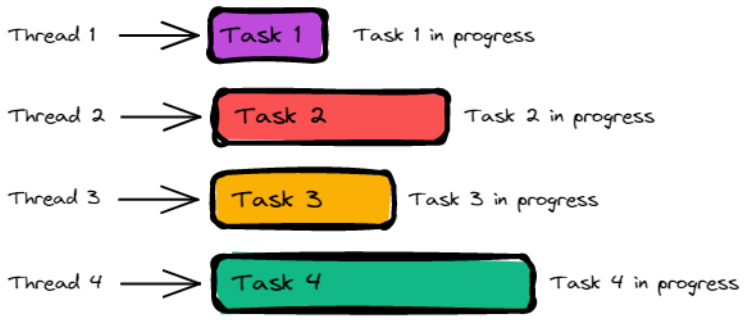
```
// Define three functions
function firstTask() {
  console.log("Task 1");
}
function secondTask() {
  console.log("Task 2");
}
function thirdTask() {
console.log("Task 3");
}
// Execute the functions
```

```
firstTask();
secondTask();
thirdTask();
```

For example, let's say that a synchronous program performs a task that requires waiting for a response from a remote server. The program will be stuck waiting for the response and cannot do anything else until the response is returned. This is known as *blocking*, and it can lead to an application appearing unresponsive or "frozen" to the user.
Consider the following code:

```
function someLongRunningFunction() {
  let start = Date.now();
  while (Date.now() - start < 5000) {
    // do nothing
  }
  return "Hello";
}
console.log('Starting...');
let result = someLongRunningFunction();
console.log(result);
console.log('...Finishing');
```

What is Asynchronous Programming?
Asynchronous programming is a way for a computer program to handle multiple tasks simultaneously rather than executing them one after the other.



Asynchronous programming allows a program to continue working on other tasks while waiting for external events, such as network requests, to occur. This approach can greatly improve the performance and responsiveness of a program.
For example, while a program retrieves data from a remote server, it can continue to execute other tasks such as responding to user inputs.
Here's an example of an asynchronous program using the setTimeout method:

```
console.log("Start of script");
setTimeout(function() {
  console.log("First timeout completed");
}, 2000);
console.log("End of script");
```

The Basics of Asynchronous programming Execution
In this , functions are executed independently of the main program flow. It means, if one task is processing, the rest of the program can continue running and executing other tasks without

waiting for asynchronous operation to finish. There are several ways in JavaScript to run program asynchronously:

- Callbacks
- Promises
- Async/ Await

Callback

Callbacks are one of the essential concepts for the asynchronous programming of JavaScript. Basically, a callback is a function that is passed as an argument to another function and is invoked or called later, after some operation has been completed. Callbacks find major use in cases where an operation will take some time to complete, for example, fetching data from a server or reading a file. Therefore , nesting of callback-based code to achieve control flow makes it hard to read and leads to phenomenon known as " callback hell ".

```
function fetchData(callback) {
  setTimeout(() => {
    callback("Data fetched successfully");
  }, 2000);
}

fetchData((data) => {
  console.log(data);
});
```

Promises

It was introduced in ES6 as a cleaner alternative to callbacks. A promise represents the eventual completion or failure of an asynchronous operation and allows chaining of multiple asynchronous operations. This chaining really makes promise-based code far more readable and manageable than callback-based code. Therefore , promises can be in one of the three states : pending , resolved(fulfilled) and rejected.

```
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve("Data fetched successfully");
    }, 2000);
  });
}
fetchData().then((data) => {
  console.log(data);
});
```

Async/Await

Async/await is a feature in modern JavaScript, which makes the writing of asynchronous code easier for both reading and maintenance. It builds on top of the concept of promises and introduces a more natural and synchronous-like way of dealing with asynchronous operations.

Basics of Async/Await

Async Functions

An async function is a special type of function that is capable of executing operations asynchronously within it. This is defined using the async keyword before a function declaration.

```
async function fetchData() {
  // Asynchronous operations go here
}
```

Await Functions

The await keyword is used inside async functions to pause the execution of the function until the Promise is resolved. It can only be used inside async functions, and it is followed by a promise. When await is used, the function execution is paused until the promise is resolved, returning its resolved value.

```
async function fetchData() {
  const data = await fetch('https://api.example.com/data');
  console.log(data);
}
```

In the above example, the async function fetchData fetches data from an API using the fetch function. The await keyword pauses the function's execution until it fetches the data from the API. Once fetched, it assigns the data to the data variable.